

Install DCGM Exporter

[DCGM Exporter](#) converts selected DCGM telemetry fields to the [Prometheus exposition format](#).

Install one exporter on each GPU node that Prometheus should monitor.

DCGM Exporter can run as a package-managed systemd service, an OCI container, or a Kubernetes DaemonSet managed by Helm or the NVIDIA GPU Operator. By default, the exporter initializes an embedded DCGM host engine inside its own process through `libdcgm`. It can instead connect to a separately managed DCGM host engine over TCP, a Unix socket, or VSOCK.

Requirements and Version Compatibility

Before installation, verify that the target system has:

- An NVIDIA GPU, driver, and DCGM combination covered by the [DCGM product support matrix](#).
- Access to the NVIDIA GPUs from the selected runtime. Container deployments require the [NVIDIA Container Toolkit](#).
- Network access from Prometheus to the exporter HTTP endpoint when Prometheus runs on another host or cluster.

See [Install DCGM](#) for DCGM package installation, privileges, and verification.

DCGM Exporter releases pair a DCGM version with an exporter version. Container image tags encode the tested pair as `<dcgm-version>--<dcgm-exporter-version>--<image-variant>`. RPM and Debian package versions identify the exporter version; `dcgm-exporter --version` reports the paired DCGM and exporter versions embedded in the executable.

! Important

Run DCGM Exporter with the DCGM version paired with that exporter release. Mismatched combinations might function, but they are not tested or supported. When troubleshooting, first update to matching, current versions of DCGM and DCGM Exporter.

For a separate DCGM host engine, the exporter's DCGM client library version must also be greater than or equal to the DCGM host engine version. Meeting this connection requirement does not make a mismatched release pair supported.

Choose an Installation Method

Method	Use it when	Lifecycle owner
Package	You manage DCGM and the exporter as services on individual hosts.	The operating system package manager and systemd.
Container	You manage a standalone container on each GPU host.	Your container runtime or service manager.
Helm	You manage the exporter directly in a Kubernetes cluster.	The DCGM Exporter Helm release.
GPU Operator	NVIDIA GPU Operator already manages the cluster's GPU software stack.	The NVIDIA GPU Operator.

Install the Package

DCGM Exporter is distributed as the `datacenter-gpu-manager-exporter` package. It installs:

- `/usr/bin/dcgm-exporter`
- `/etc/dcgm-exporter/default-counters.csv`
- `/lib/systemd/system/nvidia-dcgm-exporter.service`

Configure the NVIDIA CUDA network repository by following [the package repository guidance](#), then install or update DCGM before installing the exporter.

On Ubuntu or Debian:

```
$ sudo apt-get install datacenter-gpu-manager-exporter
```

On Red Hat Enterprise Linux or a compatible distribution:

```
$ sudo dnf install datacenter-gpu-manager-exporter
```

Start the standalone DCGM host engine and the exporter:

```
$ sudo systemctl enable --now nvidia-dcgm
$ sudo systemctl enable --now nvidia-dcgm-exporter
```

Configure the systemd service

The package unit starts after `nvidia-dcgm.service`, starts the exporter in embedded mode with the default collector CSV, and restarts after a failure. Inspect the effective unit, including administrator overrides:

```
$ systemctl cat nvidia-dcgm-exporter.service
```

Do not edit the vendor unit under `/lib` because a package upgrade can replace it. Create an administrator drop-in instead:

```
$ sudo systemctl edit nvidia-dcgm-exporter.service
```

For example, replace the packaged command to connect to a separate DCGM host engine:

```
[Service]
ExecStart=
ExecStart=/usr/bin/dcgm-exporter \
  -f /etc/dcgm-exporter/default-counters.csv \
  -r tcp://host123:5555
```

The empty `ExecStart=` clears the vendor command before the replacement is defined. Restart and inspect the service:

```
$ sudo systemctl restart nvidia-dcgm-exporter.service
$ systemctl status nvidia-dcgm-exporter.service --no-pager
$ journalctl --unit nvidia-dcgm-exporter.service --since today
```

After an exporter package upgrade, compare a replaced `ExecStart` with the updated vendor command because the drop-in does not inherit vendor command changes. See [systemd unit layering](#) for drop-in precedence and directive behavior.

Run the Container

Use `latest` to follow the current exporter release, or choose a current semantic-version tag from the [DCGM Exporter container catalog](#) to control upgrades. Set `DCGM_EXPORTER_TAG` to the selected tag, then run:

```
$ export DCGM_EXPORTER_TAG=latest
$ docker run -d --rm \
  --name dcgm-exporter \
  --gpus all \
  --cap-add SYS_ADMIN \
  -p 9400:9400 \
  nvcr.io/nvidia/k8s/dcgm-exporter:${DCGM_EXPORTER_TAG}
```

This command exposes every GPU and allows DCGM profiling fields such as `DCGM_FI_PROF_*`. Review Docker's [runtime privilege and Linux capabilities](#) before adapting the command to a production host. Access to the Docker daemon also requires administrative authorization.

Install with Helm

The Helm chart deploys one exporter pod on each selected GPU node and enables Kubernetes pod mapping. Install [Helm](#), then add the chart repository:

```
$ helm repo add gpu-helm-charts https://nvidia.github.io/dcgm-exporter/helm-charts
$ helm repo update
```

If the cluster has the Prometheus Operator custom resource definitions, install the chart with its default `ServiceMonitor`:

```
$ helm install dcgm-exporter gpu-helm-charts/dcgm-exporter \
  --namespace gpu-monitoring \
  --create-namespace
```

Otherwise, disable the `ServiceMonitor` until those definitions are installed:

```
$ helm install dcgm-exporter gpu-helm-charts/dcgm-exporter \
  --namespace gpu-monitoring \
  --create-namespace \
  --set serviceMonitor.enabled=false
```

Wait for the DaemonSet and inspect its pods:

```
$ kubectl rollout status daemonset/dcgm-exporter \
  --namespace gpu-monitoring
$ kubectl get pods \
  --namespace gpu-monitoring \
  --selector app.kubernetes.io/name=dcgm-exporter
```

Install with GPU Operator

The [NVIDIA GPU Operator](#) can manage the driver, container toolkit, DCGM, DCGM Exporter, and related Kubernetes resources as one GPU software stack. Use this method when the Operator already owns those components.

Follow the GPU Operator installation procedure, then verify that its exporter pods are running:

```
$ kubectl get pods --all-namespaces \
  --selector app=nvidia-dcgm-exporter
```

Configure custom fields through the Operator-managed metrics ConfigMap. Do not edit the Operator-owned DaemonSet directly.

Configure DCGM Exporter

Command-line flags and environment variables explicitly supplied at startup override values loaded from an optional YAML file. A collector CSV or the YAML `metrics` section selects the fields to export. See the comprehensive [dcgm-exporter command reference](#) for every flag, environment variable, default, and validation rule.

Connect to a DCGM Host Engine

Omitting `--remote-hostengine-info` uses an embedded DCGM host engine. To connect to a separately managed DCGM host engine, use one of these forms:

- `host:port`
- `tcp://host:port`
- `unix:///path/to/socket`
- `vsock://cid:port`
- `[ipv6-address]:port`

The URI forms require DCGM 4.5 or later. Examples:

```
$ dcgm-exporter -r tcp://host123:5555
$ dcgm-exporter -r unix:///tmp/nv-hostengine
$ dcgm-exporter -r vsock://3:5555
$ dcgm-exporter -r "[::1]:5555" -a "[::]:9400"
```

The endpoint must match the transport and listener configured for the DCGM host engine. A container also needs access to the corresponding host network, Unix socket, or VSOCK device.

Select Metrics

A collector CSV has exactly three columns:

```
DCGM field, Prometheus metric type, help message
```

Supported types are `counter`, `gauge`, `untyped`, and `label`. For a metric row, the help message becomes the Prometheus `# HELP` text and does not affect collection. A `label` row adds a returned string value to other samples instead of creating a metric family; its help message is not emitted.

Create a custom collector:

```
DCGM_FI_DEV_GPU_UTIL, gauge, GPU utilization (in %).
DCGM_FI_DEV_FB_USED, gauge, Framebuffer memory used (in MiB).
DCGM_FI_SYSTEM_DRIVER_VERSION, label, Driver version.
```

Pass it with `--collectors`. For a container, mount the file read-only and pass its in-container path:

```
$ docker run -d --rm \
  --name dcgm-exporter \
  --gpus all \
  --cap-add SYS_ADMIN \
  -p 9400:9400 \
  --mount type=bind,src=/opt/dcgm-exporter/custom.csv,dst=/etc/dcgm-exporter/custom.csv \
  nvcr.io/nvidia/k8s/dcgm-exporter:${DCGM_EXPORTER_TAG} \
  --collectors /etc/dcgm-exporter/custom.csv
```

In Helm, `customMetrics` replaces rather than extends the shipped collector CSV. See [DCGM Exporter Metrics](#) for the shipped inventory, exporter-owned metrics, label behavior, and custom-field constraints.

Set Collection Cadence

`--collect-interval` sets the default DCGM watch cadence requested by the exporter, in milliseconds; the default is 30000. YAML accepts durations and can assign different field patterns to watch groups:

```
version: 1
metrics:
  file: /etc/dcgm-exporter/default-counters.csv
collection:
  interval: 30s
watchGroups:
  - name: fast-thermals
    interval: 5s
    fields:
      - DCGM_FI_DEV_GPU_TEMP_CELSIUS
      - DCGM_FI_DEV_BOARD_POWER_WATTS
```

DCGM Exporter reads YAML only at startup. The active file-backed collector CSV is watched for changes and can also be reloaded with `SIGHUP`. See [Fields and Watches](#) for how DCGM combines watches and caches samples. The DCGM host engine refreshes that cache at the effective watch

cadence. Each HTTP scrape reads the latest cached sample; the exporter does not have a separate periodic capture or file-storage interval.

Select Devices and Entities

Use `--devices` for GPUs and GPU instances, `--switch-devices` for NVSwitches and NVLinks, and `--cpu-devices` for CPUs and CPU cores. The default `f` selector monitors each full GPU without MIG and each GPU instance with MIG. The command reference documents the complete `f`, `g`, `i`, range, and combined-selector grammar.

Add Workload Labels

Enable only the metadata required by queries and alerts because each unique label set creates another Prometheus time series.

Optional workload mappings

Mapping	Primary setting	Result
Kubernetes pods	<code>--kubernetes</code>	Adds pod, namespace, and container labels from the kubelet pod-resources socket.
Pod labels and UID	<code>--kubernetes-enable-pod-labels</code> and <code>--kubernetes-enable-pod-uid</code>	Adds selected pod metadata. Restrict labels with the allowlist regex.
Shared or virtual GPUs	<code>--kubernetes-virtual-gpus</code>	Adds virtual-GPU attribution for supported sharing strategies.
Kubernetes DRA	<code>--kubernetes-enable-dra</code>	Adds claim, driver, pool, device, and applicable MIG labels.
Runtime containers	<code>--container-labels</code>	Maps explicit GPU assignments through a mounted Docker-compatible runtime socket outside Kubernetes.
HPC jobs	<code>--hpc-job-mapping-dir</code>	Adds job IDs from scheduler-managed GPU mapping files.

Verify the Installation

A successful metrics response contains names that begin with `DCGM_`:

```
$ curl --fail http://localhost:9400/metrics
```

For Kubernetes, forward the Service before running the same command:

```
$ kubectl port-forward \  
  --namespace gpu-monitoring \  
  service/dcgm-exporter 9400:9400
```

Administer and Troubleshoot the Exporter

Restart the package service with `systemctl restart nvidia-dcgm-exporter.service`. Replace a standalone container with the same configuration to upgrade it. Upgrade a Helm release after refreshing the repository:

```
$ helm repo update  
$ helm upgrade dcgm-exporter gpu-helm-charts/dcgm-exporter \  
  --namespace gpu-monitoring \  
  --reuse-values
```

Changing a collector CSV reloads metric selection. Changing YAML requires a restart. A restart or reload resets exporter-owned counters stored in process memory and can briefly interrupt the endpoint.

Use `--web-config-file` with a Prometheus exporter-toolkit configuration to enable TLS or basic authentication. Protect runtime sockets, debug dumps, and profiling endpoints because they can expose host, process, or workload data. `--enable-pprof` requires a web configuration so the profiling endpoints are protected.

When the endpoint or expected metrics are unavailable:

1. Run `nvidia-smi` on the GPU host and confirm that the exporter process, container, or DaemonSet is running.
2. Inspect the exporter logs and verify that port `9400` is published and reachable.
3. For a separate DCGM host engine, verify the endpoint, transport, socket mount, and client/host-engine version relationship.
4. Confirm that the field is selected, then follow its link in the metrics reference for entity and runtime support.
5. For `DCGM_FI_PROF_*` fields, verify GPU support and the required `SYS_ADMIN` capability.
6. For Kubernetes labels, verify the pod-resources socket, device ID type, service account, and RBAC.

Use `--debug` and structured logging for more detail. The command reference documents DCGM logging, debug dumps, and pprof. For broader DCGM failures, see [Debugging and Troubleshooting](#).

Related Software

Software related to DCGM Exporter		
Component	Relationship	Required for
NVIDIA driver and NVML	Provide GPU access and the management library used by DCGM. NVML is installed with the driver.	Every deployment.
nvidia-smi	Verifies that the driver can discover the GPUs and provides a first diagnostic check before troubleshooting DCGM Exporter.	Host verification and troubleshooting.
DCGM	Discovers entities, owns field watches, and supplies cached telemetry.	Every deployment.
NVIDIA Container Toolkit	Exposes NVIDIA GPUs to containers.	Container deployments.
NVIDIA GPU Operator	Manages the Kubernetes GPU software stack and exporter lifecycle.	Operator-managed clusters.
Helm	Installs and upgrades the standalone DCGM Exporter Kubernetes chart.	Helm-managed clusters.
Prometheus Operator	Supplies the <code>ServiceMonitor</code> custom resource used by the chart's default scrape configuration.	Chart-managed <code>ServiceMonitor</code> resources.
Prometheus	Scrapes, stores, and queries the exported time series.	Central metric collection.

Next Steps



Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.